UNIVERSIDAD EUROPEA DEL ATLÁNTICO

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA INFORMÁTICA

TRABAJO FINAL DE GRADO

AUTOMATIZACIÓN DEL TESTING MEDIANTE AGENTES DE INTELIGENCIA ARTIFICIAL PARA LA GENERACIÓN DE ESCENARIOS DE PRUEBA

Trabajo fin de grado realizado por

DAVID GARCÍA COSTA

bajo la dirección de

LORENZO PÉREZ BRITO

SANTANDER – 14 - 03 - 2026

EUROPEAN UNIVERSITY OF THE ATLANTIC

HIGHER POLYTECHNIC SCHOOL

DEGREE IN COMPUTER ENGINEERING

DISSERTATION

AUTOMATION OF SOFTWARE TESTING USING ARTIFICIAL INTELLIGENCE AGENTS FOR TEST SCENARIO GENERATION

Dissertation carried out by

DAVID GARCÍA COSTA

supervised by

LORENZO PÉREZ BRITO

SANTANDER – 14 - 03 - 2026

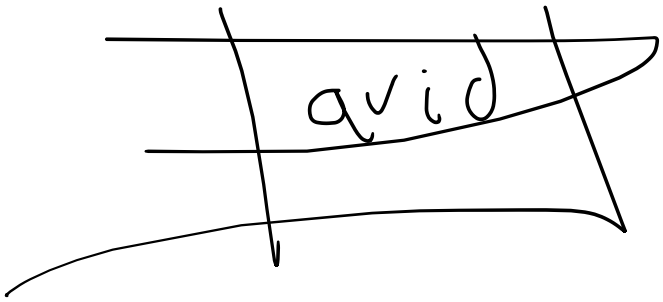
DECLARACIÓN DE ORIGINALIDAD DEL TFG

D./Dña. David García Costa, con DNI (NIE o pasaporte) 51536821P, declaro que el presente Trabajo Fin de Grado es original y todas las fuentes de información han sido debidamente citadas, no habiendo incurrido en plagio, según lo estipulado en la normativa general para los Trabajos de Fin de Grado de la Universidad Europea del Atlántico.

Y para que así conste, lo suscribo y firmo.

En Santander, a 14 de marzo de 2026

Firma del alumno



Agradecimientos

Esta sección es opcional. En esta sección, el autor agradece a las personas o instituciones que colaboraron en la realización del trabajo. Si se incluye esta sección, deberán aparecer los nombres completos, los cargos y su aporte al trabajo.

Resumen

Este Trabajo de Fin de Grado aborda una propuesta para automatizar una parte del trabajo de QA en CIC Consulting Informático mediante el uso de agentes de Inteligencia Artificial. Antes de probar un sistema, existe una fase previa de análisis y preparación que toma mucho tiempo y depende en gran medida de la experiencia del ingeniero de QA.

A partir de esa necesidad, en este trabajo se plantea una solución capaz de leer documentación, identificar la información relevante y convertirla en escenarios de prueba. Para ello, se diseña una arquitectura multiagente que reparte funciones entre distintos componentes especializados, con el fin de apoyar y hacerlo más eficiente.

La propuesta se desarrolla con Google ADK, un modelo de OpenAI y la integración con Kiwi TCMS mediante API.

En conjunto, este trabajo presenta una aplicación práctica de la Inteligencia Artificial en el ámbito del testing de software y analiza hasta qué punto este enfoque puede mejorar la preparación y organización de los casos de prueba.

Palabras clave: QA, testing, automatización, Inteligencia Artificial, agentes de IA.

Abstract

This Final Degree Project presents a proposal to automate part of the QA workflow at CIC Consulting Informático through the use of Artificial Intelligence agents. Before testing a system, there is a preliminary phase of analysis and preparation that requires significant time and relies heavily on the experience of the QA engineer.

Based on this need, the project proposes a solution capable of reading documentation, identifying relevant information, and transforming it into test scenarios. To achieve this, a multi-agent architecture is designed in which different specialized components perform specific tasks in order to support the process and improve its efficiency.

The proposal is developed using Google ADK, an OpenAI model, and integration with Kiwi TCMS through its API.

Overall, this work presents a practical application of Artificial Intelligence in the field of software testing and analyzes the extent to which this approach can improve the preparation and organization of test cases.

Keywords: QA, testing, automation, Artificial Intelligence, AI agents.

Índice

Generar índice de manera automática mediante la aplicación de formato de párrafo y estilos a los epígrafes y subepígrafes y la función «Insertar Tabla de contenido» (Pestaña de Referencias).

1. Introducción 1
2. Marco teórico 2
2.1. Justificación 13
2.2. Solución propuesta 15
2.3. Objetivos General y Específicos 15
2.4. Estructura del trabajo 16
3. Requerimientos y requisitos 18
4. Análisis y diseño 19
5. descripción de la solución propuesta 20
6. Conclusiones 21
7. Referencias bibliográficas 22
8. Anexos 24

Índice de figuras

Generar de manera automática con la función de «Insertar título» (Pestaña de Referencias) a las figuras e «Insertar Tabla de ilustraciones» (Pestaña de Referencias).

1. INTRODUCCIÓN

El capítulo de Introducción se entiende como una breve presentación del tema del TFG, una idea somera pero concisa de los diferentes aspectos de los que se compone el trabajo, así como de las implicaciones que puede tener en la comunidad científica o en la práctica profesional.

Motivación

En este Trabajo de Fin de Grado se propone realizar un diseño e implementar un sistema de agentes de Inteligencia Artificial orientado a mejorar el proceso de testing de QA de CIC Consulting Informático.

La motivación que lleva a realizar este trabajo surge de una necesidad identificada en CIC Consulting Informático.

Actualmente, el esfuerzo de los ingenieros de QA está en hacer tareas de elevada carga; como puede ser la interpretación de documentación (Documentos de Requisitos Funcionales y Documentos de Diseño del Sistema), el análisis de los requisitos funcionales y casos de uso que contienen dichos documentos, la redacción manual de casos de prueba dentro de un Excel, en KiwiTCMS ahora mismo solo hay unos cuantos ejemplos de los escenarios en lenguaje Gherkin y están dentro de una carpeta de VSCode donde trabaja el ingeniero, no están en ninguna API. Estas actividades requieren considerable inversión de tiempo y conocimiento especializado, lo que limita la escalabilidad del proceso y genera dependencia.

Problemática Identificada

En términos más específicos, se puede decir que el flujo de trabajo actual en CIC presenta las siguientes limitaciones:

- Dependencia del análisis manual

- Falta de automatización en fase de análisis

- Escalabilidad limitada

Propuesta de Solución

Para resolver este problema, el presente trabajo propone el diseño de una arquitectura basada en agentes de Inteligencia Artificial, utilizando Google ADK con un modelo de OpenAI, que permita automatizar la generación de casos de prueba a partir de la documentación existente.

La solución se estructura en torno a un sistema multiagente especializado, cuyos componentes se detallarán en capítulos posteriores, donde cada agente asumirá un rol específico dentro del flujo de trabajo.

Esta arquitectura aprovecha las capacidades de los agentes de IA no solo para ejecutar tareas repetitivas, sino también para interpretar y analizar la documentación por parte de los Project Managers y Clientes, estructurar la información y tomar decisiones en un proceso colaborativo orientado a un objetivo común.

2. MARCO TEÓRICO

El capítulo Marco teórico engloba una revisión del estado de la cuestión sobre el tema estudiado en el TFG y que sirve, a su vez, para fundamentar el trabajo.

Para ello, se podrá realizar mediante consulta en bases de datos especializadas, contraste de información, análisis histórico del tema, etc.

Estado del Arte

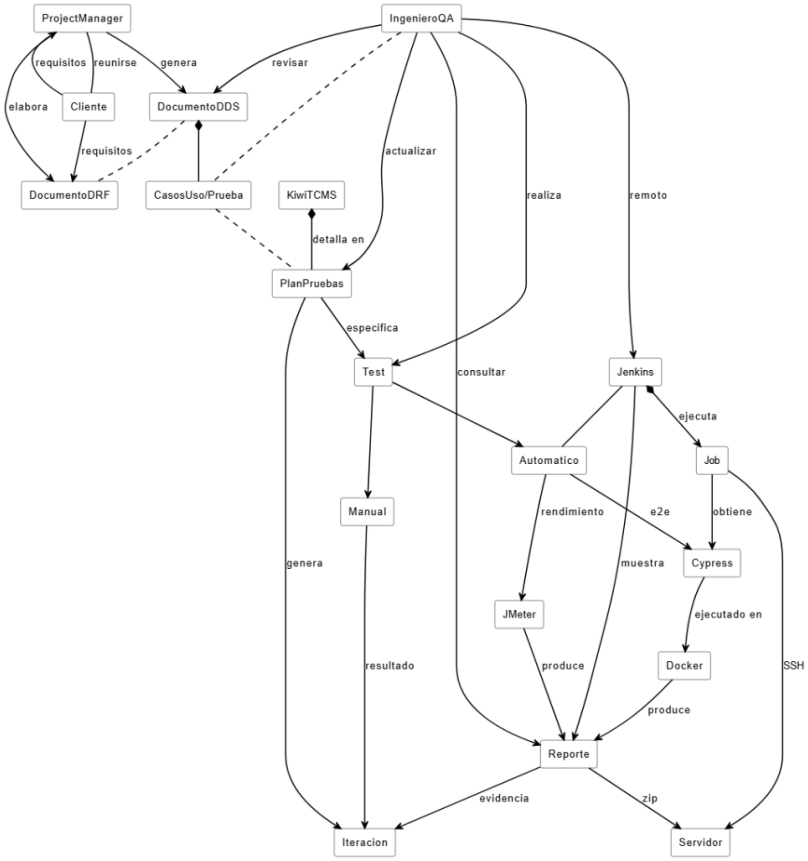
Contexto empresarial de CIC Consulting Informático

CIC Consulting Informático es una empresa de ingeniería y desarrollo de proyectos de informática y comunicaciones (CIC Consulting Informático, s. f.).

En el proceso actual de QA en CIC, el trabajo comienza con una reunión por parte del cliente con el Project Manager, en esa reunión se recogen los requisitos del producto y los casos de uso que debería tener; esto se plasma sobre documentación, en este caso utilizan DRF (Documentos Requisitos Funcionales) y DDS (Documento Diseño Sistema).

A partir de esta información, el ingeniero de QA define casos de prueba que actualmente están en un Excel y algunos de ellos se referencian en una carpeta dentro de VSCode con estructuras Gherkin propio de Kiwi TCMS, pero no se usa ninguna API, ejecuta las pruebas end-to-end en Cypress y documenta incidencias o resultados en las herramientas correspondientes.

Este flujo de trabajo se apoya en herramientas ya existentes detalladas en la foto que se presenta a continuación y en la experiencia del personal de QA, pero mantiene una importante dependencia del análisis manual y de la intervención directa del ingeniero.



Por lo que para el ingeniero de QA supone un gran esfuerzo interpretar correctamente la documentación de entrada y traducirla a elementos útiles para validar el producto.

A partir del contexto descrito, se pretende mejorar el proceso de QA en CIC Consulting Informático mediante una solución capaz de asistir el análisis de documentación, estructurar requisitos y casos de uso, y transformarlos en escenarios de prueba útiles para la gestión del testing.

No se trata simplemente de automatizar pruebas en sentido amplio, sino de intervenir en una fase previa del flujo, actualmente con mucho esfuerzo manual, y conocimiento experto.

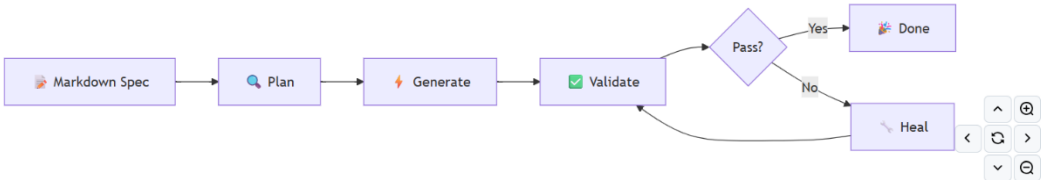
En consecuencia, el trabajo se centra en estudiar cómo una arquitectura basada en agentes de Inteligencia Artificial puede analizar documentos DRF y DDS, extraer información funcional relevante (requisitos funcionales y casos de uso), transformarlo a escenarios Gherkin y registrar los escenarios (Given, When, Then, And) en casos de prueba en Kiwi TCMS a través de su API.

Solución Existente Aproximada

Quorvex AI

En el testing asistido por Inteligencia Artificial, han comenzado a aparecer soluciones que no se limitan a ejecutar pruebas automatizadas, también intervienen en su construcción a partir de descripciones funcionales en lenguaje natural. Una de las propuestas más próximas al enfoque de este trabajo es Quorvex AI, es una herramienta orientada a la generación automática de tests a partir de especificaciones redactadas en lenguaje natural (NihadMemmedli, s. f.).

El interés de esta solución reside en que plantea un modelo de trabajo en el que la IA no actúa solo como generador de texto, sino como un componente capaz de participar en varias fases del proceso de testing. Al empezar con la especificación inicial, el sistema analiza la información proporcionada, genera un plan de ejecución, construye la prueba correspondiente y por último lo valida sobre la aplicación real. Si detecta errores durante la ejecución, la propia herramienta puede corregir automáticamente parte del código generado con una función de autorreparación (NihadMemmedli, s. f.).



Quorvex AI se apoya principalmente en Playwright como framework de automatización y en modelos de lenguaje para interpretar la especificación y la transforma en código ejecutable. Esto le permite convertir descripciones escritas en lenguaje natural en pruebas end-to-end para que las ejecute, esto reduce parte del esfuerzo manual que normalmente requiere la construcción inicial de los tests (NihadMemmedli, s. f.).

Pero, por otra parte, esta solución presenta limitaciones. En primer lugar, el enfoque está claramente orientado a la generación de tests en Playwright, lo que implica una dependencia fuerte de este framework. Y aunque automatiza la construcción y repara pruebas, no está tan centrada en la gestión documental previa ni en la integración con herramientas de gestión de

pruebas como Kiwi TCMS, que sí constituyen una parte importante de la propuesta desarrollada en este trabajo.

Por tanto, Quorvex AI puede considerarse como una solución existente y relevante, especialmente por su capacidad para transformar especificaciones naturales en artefactos de testing automatizado. Pero su objetivo se sitúa en una fase distinta del proceso. Mientras Quorvex AI se orienta a la generación directa de tests end-to-end ejecutables, la propuesta de este TFG se centra en una etapa previa, aunque de cierta manera y en líneas futuras se asemeja.

MARCO TEÓRICO

Contexto general de la calidad del software

El desarrollo de software de baja calidad puede generar incidencias, retrasos y costes adicionales en el ciclo de vida del sistema (Sharma & Kumar, 2024).

La calidad del software no se limita a que una aplicación funcione, sino que se haga de manera correcta y que siga la línea de los requisitos y casos de uso definidos previamente.

Entonces, el asegurar la calidad, por definición representa el conjunto de procesos, prácticas y mecanismos que garantizan que el software se desarrolla y se valida en base a unos estándares definidos (Sharma & Kumar, 2024). Este previene defectos, da fiabilidad y evidencia que el sistema cumple con la calidad esperada, es decir, da una capa de control y de mejora continua.

Papel del testing

El testing constituye una de las actividades más relevantes dentro de QA. Su función principal se basa en diseñar, ejecutar y evaluar pruebas con el fin de comprobar si un sistema se comporta según lo esperado. A través del testing se validan los requisitos, se detectan los defectos y todo ello antes de la puesta en producción y eso otorga confianza sobre el estado del producto (Sharma & Kumar, 2024).

Se puede decir que el testing tradicional tiene limitaciones, ya que muchas veces el proceso depende de un esfuerzo manual para interpretar la documentación, diseñar casos de prueba, ejecutar validaciones y registrar incidencias, lo que incrementa el tiempo necesario y la carga

operativa del equipo (Wang, Zhang, & Li, 2024). Por lo que la repetición de tareas similares y depender del conocimiento del ingeniero de QA hace que sea muy difícil poder escalar el proceso y puede limitar tanto la cobertura como la capacidad de adaptación a nuevas necesidades.

Automatización del testing

La automatización del testing permite ejecutar validaciones de forma repetible, rápida y consistente, haciendo uso de scripts, herramientas y pipelines que permiten reducir parte de la intervención manual y aportar estabilidad (Selenium Project, s. f.; Cypress, s. f.).

No obstante, sigue siendo necesario interpretar requisitos y diseñar los casos de prueba, y el testing tradicional no siempre resuelve el problema previo.

Aplicación de IA

La aplicación de Inteligencia Artificial al testing supone una evolución. No solo puede ejecutar o asistir tareas repetitivas, sino también participar en procesos de análisis, estructuración y generación de contenido útil para QA, como escenarios de prueba o interpretaciones semánticas de documentación (Wang, Zhang, & Li, 2024).

Este cambio de enfoque resulta especialmente interesante cuando la carga del proceso recae sobre tareas cognitivas y documentales. En ese tipo de contexto, la IA puede aportar apoyo no tanto en la ejecución, sino en la fase de preparación del conocimiento para el testing.

El AI-driven testing hace referencia a enfoques en los que modelos de IA intervienen en la generación, priorización, análisis o mantenimiento de pruebas. A diferencia de las aproximaciones tradicionales, aquí la IA participa en tareas que requieren cierto grado de interpretación y adaptación.

Una de las aplicaciones más relevantes de la IA en testing es la generación automática o asistida de casos de prueba a partir de requisitos, historias de usuario o documentación funcional. Esta capacidad permite reducir parte del esfuerzo manual y aumentar la velocidad con la que se construyen escenarios iniciales de validación.

Sin embargo, la calidad del resultado depende de cómo se interprete la entrada, de la consistencia del modelo empleado y del grado de control que se mantenga sobre la transformación. Por ello, la generación automática no debe entenderse como un proceso

completamente trivial o garantizado, sino como una línea de apoyo que requiere diseño y validación.

Los agentes de IA

Estos representan una evolución respecto al uso de modelos generativos.

Un agente no solo produce una respuesta de texto, sino que percibe la información, toma decisiones, utiliza herramientas y ejecuta acciones que están orientadas a un objetivo concreto (Russell & Norvig, 2021).

Esto hace que los agentes sean idóneos para procesos con varias etapas conectadas entre sí. En este trabajo, el análisis de documentación, la transformación a Gherkin y la integración con una API son tareas que encajan muy bien en una arquitectura de este tipo.

Definición de agente

Un agente puede definirse como una entidad software que percibe información de su entorno, la procesa, toma decisiones y ejecuta acciones en función de un objetivo.

En términos prácticos, un agente puede combinar un modelo de lenguaje con instrucciones, herramientas, memoria y mecanismos de control, lo que le permite abordar tareas más complejas que una simple generación de texto.

Capacidades de percepción, decisión y acción

Las capacidades fundamentales de un agente pueden resumirse en percibir, decidir y actuar. Percibir implica recoger información relevante del entorno o de los datos de entrada. Decidir supone interpretarla y seleccionar una estrategia adecuada. Actuar significa ejecutar una operación útil sobre una herramienta, un sistema o un flujo de trabajo.

Estas capacidades son especialmente valiosas cuando se desea automatizar procesos encadenados. En este caso, por ejemplo, permiten pasar de un documento de entrada a una salida estructurada y utilizable dentro de una herramienta de gestión de pruebas.

Diferencia entre IA generativa e IA agentica

La IA generativa tradicional responde a una entrada mediante la producción de una salida, normalmente textual (Wang, Zhang, & Li, 2024).

La IA agentica incorpora el uso de herramientas, pasos intermedios y la capacidad de verificación o control del flujo. Por ello, esta última resulta más adecuada para procesos donde se necesita algo más que una respuesta aislada.

Esto resulta relevante porque la propuesta del TFG no se limita a pedir al modelo que por ejemplo "redacte" escenarios, sino que requiere una arquitectura capaz de procesar los documentos, estructurar esa información, transformarla y registrarla en una herramienta externa.

Una característica clave de los agentes es su capacidad para utilizar herramientas externas, como APIs, sistemas de almacenamiento, navegadores o conectores.

En la propuesta de este TFG, esta capacidad es clave, ya que la integración con Kiwi TCMS mediante API constituye una parte central del flujo. Sin uso de herramientas, la solución quedaría limitada a una simple generación documental sin conexión operativa.

Por otra parte, para operar de forma coherente, un agente necesita gestionar el contexto, el estado del proceso y, en ciertos casos, mecanismos de memoria. Esto le permite mantener consistencia entre etapas, recordar información relevante y actuar de forma alineada con el objetivo definido.

En sistemas donde varias tareas se enlazan entre sí, esta gestión resulta especialmente importante. Analizar un documento, estructurar requisitos y convertirlos en escenarios exige conservar relación entre la información original y la salida generada.

Sistemas multiagente

Cuando un único agente no es suficiente para abordar todas las tareas necesarias, puede recurrirse a un sistema multiagente.

Es decir, varios agentes que están programados para realizar tareas concretas colaboran entre sí para resolver un problema común, repartiéndose funciones y manteniendo una coordinación explícita.

Un sistema multiagente puede definirse como una arquitectura compuesta por varios agentes que colaboran, se coordinan y comparten información para resolver un problema (Russell & Norvig, 2021).

Coordinación entre agentes

En cuanto a la coordinación no basta con tener varios componentes especializados; se necesita establecer cómo se comunican, en qué orden actúan, qué información comparten y cómo se asegura la coherencia del proceso.

En la práctica, esta coordinación puede realizarse con un componente orquestador o con reglas de flujo definidas.

Especialización de roles

Es una gran ventaja, ya que puede existir un agente centrado en analizar documentos, otro en estructurar requisitos, otro en transformar la información a Gherkin y otro en integrarla con una API externa a través de herramientas.

Soluciones tradicionales de automatización y testing

1. Selenium

Selenium se ha consolidado como una de las referencias históricas en automatización de navegadores (Selenium Project, s. f.). Su principal fortaleza es la flexibilidad y el amplio ecosistema que lo rodea, lo que lo convierte en una herramienta muy extendida para pruebas web.

Este requiere de un esfuerzo notable de configuración y mantenimiento.

Pero eso hace que, no resuelva por sí mismo los problemas de análisis documental y estructuración de escenarios.

2. Cypress

Cypress es una herramienta moderna orientada a pruebas end-to-end y de componentes en aplicaciones web (Cypress, s. f.). Destaca por su integración con el entorno de desarrollo, su facilidad de uso y su capacidad para observar la ejecución de pruebas de forma visual.

En el contexto de CIC, Cypress forma parte del ecosistema real de trabajo. En el presente TFG no se llegará a implementar, pero sí que se quiere implementar a nivel de empresa.

3. JUnit y TestNG

Son marcos de prueba ampliamente utilizados en el ecosistema Java (JUnit Team, s. f.; TestNG, s. f.) para la definición y ejecución de pruebas unitarias e integradas. Aportan

estructura, anotaciones y capacidad de organización, siendo herramientas muy habituales en procesos de desarrollo con fuerte componente backend.

4. Playwright

Playwright ha ganado relevancia por su enfoque moderno de automatización web (Playwright Team, s. f.), su soporte para múltiples navegadores y sus capacidades de depuración y espera automática.

Estas características lo convierten en una opción muy atractiva para pruebas sobre interfaces dinámicas.

Aun así, como ocurre con otras herramientas de automatización, su foco principal está en la ejecución de pruebas, no en la fase previa de interpretación documental y construcción estructurada de escenarios.

5. Kiwi TCMS

Kiwi TCMS es una herramienta de gestión de pruebas de código abierto que permite definir casos de prueba, organizarlos en planes de prueba, registrar sus ejecuciones y mantener la trazabilidad de todo el proceso de validación (Kiwi TCMS, s. f.). Dentro de este contexto, un scenario es una manera de describir un caso de prueba como una situación concreta que se quiere comprobar en el sistema.

Para que esa descripción sea más clara, natural y fácil de seguir, suele utilizarse la estructura Given / When / Then / And, propia del enfoque BDD (Behavior Driven Development). Given define el contexto inicial o las condiciones previas, When indica la acción que realiza el usuario o el sistema, y Then expresa el resultado esperado. La palabra And permite añadir información o pasos adicionales sin romper la estructura. Esta forma de redactar los casos de prueba hace que sean mucho más comprensibles, tanto para perfiles de testing como para desarrollo, y facilita una comunicación más clara entre todos los implicados.

Su interés en este trabajo es especialmente alto porque puede actuar como punto de destino de los escenarios generados por el sistema multiagente. Además, el hecho de contar con API facilita su integración con arquitecturas basadas en herramientas y automatización, lo que la convierte en un elemento especialmente adecuado para la propuesta concreta de este TFG.

Uno de los agentes se encarga de conectarse con la API de Kiwi TCMS para insertar automáticamente la información generada en la herramienta (salida) (A partir de los escenarios, este agente crea los casos de prueba dentro de la plataforma)

Así la solución no se limita a generar texto de forma aislada, sino que transforma la salida generada por un agente en elementos útiles dentro del proceso de QA. Esto permite mantener la información organizada para que el equipo de QA lo utilice.

Herramientas de testing basadas en IA

1. Testim

Testim es una plataforma orientada a la automatización de pruebas con capacidades de IA que ayudan a mejorar la estabilidad y reducir el mantenimiento (Testim, s. f.).

Su propuesta se centra en absorber parte del impacto de cambios frecuentes en la interfaz.

2. Applitools

Applitools se orienta a la detección de regresiones visuales mediante Visual AI, permitiendo identificar diferencias perceptibles para el usuario en distintos navegadores y dispositivos (Applitools, s. f.).

Este enfoque resulta especialmente potente en validaciones visuales complejas.

3. Mabl

Mabl busca unificar creación, ejecución y análisis de pruebas web y API dentro de entornos guiados, incorporando capacidades de IA para hacer el proceso más asistido (mabl, s. f.).

Su enfoque facilita determinadas tareas operativas y de mantenimiento.

4. Functionize

Functionize plantea una automatización empresarial apoyada en IA y capacidades cercanas a enfoques agénticos para acelerar la creación, mantenimiento y ejecución de flujos end-to-end (Functionize, s. f.).

Frameworks para el desarrollo de agentes

1. Google ADK

Google ADK es un framework orientado al desarrollo de agentes de IA, con capacidad para construir soluciones modulares, estructuradas y orientadas a procesos. Su interés está en la facilidad para crear arquitecturas en las que varios componentes colaboran dentro de un flujo definido. En su documentación se explica todo muy detalladamente y es fácil de seguir (Google, s. f.).

2. LangGraph

LangGraph se orienta a construir aplicaciones stateful y multi-actor basadas en grafos, con especial atención a la persistencia, la orquestación y la trazabilidad de flujos largos. Su diseño resulta interesante para procesos en los que el estado y la secuencia de los pasos son críticos (LangChain, s. f.).

3. AutoGen

AutoGen ha sido una referencia importante en el desarrollo de aplicaciones basadas en conversaciones entre agentes y en la experimentación con arquitecturas colaborativas.

Sin embargo, este framework en concreto dependiendo del tipo de solución, puede resultar menos alineado que otros enfoques más orientados a procesos (Microsoft, s. f.).

4. CrewAI

CrewAI se centra en la orquestación de agentes colaborativos y flujos complejos, ofreciendo abstracciones de alto nivel para modelar equipos de agentes con roles definidos. Esta orientación resulta útil cuando se desea construir sistemas con separación clara de responsabilidades (CrewAI, s. f.).

Criterio	**Google ADK**	**LangGraph**	**AutoGen**	**CrewAI**
Soporte multiagente	SI	SI	SI	SI
Flujos estructurados / orientados a proceso	SI	SI	NO	SI

Gestión fuerte de estado / persistencia	NO	SI	NO	SI
Enfoque principalmente conversacional	NO	NO	SI	NO
Encaje directo con este TFG	SI	SI	NO	SI

2.1. JUSTIFICACIÓN

Este apartado presenta una exposición de situación a abordar y las razones que validan la realización de nuestro estudio.

En primer lugar, se puede decir que las soluciones tradicionales de automatización de testing no resuelven de forma completa el problema planteado.

El problema de este TFG está orientado al hecho de cómo construir las pruebas. El análisis de la documentación que te dan los Project Managers, la extracción de requisitos y casos de uso relevantes y su transformación a otro lenguaje son tareas con una elevada carga.

Por eso el enfoque se centra en asistir en la comprensión, estructuración y preparación de la información necesaria para el testing.

El hecho de aplicar Inteligencia Artificial aporta un gran valor en este punto, ya que permite intervenir en procesos como la interpretación de documentación, la estructuración de la información de esos documentos y la generación de contenido. En particular, la posibilidad de generar casos de prueba a partir de requisitos o documentación funcional supone una gran ventaja a la hora de reducir parte del esfuerzo manual y acelerar la construcción de los casos de prueba.

Por otra parte, hay que tener en cuenta que la calidad del resultado depende de la forma en que el agente interprete la entrada, de la consistencia del modelo utilizado y del grado de control que se mantenga.

Por ello, la propuesta no puede apoyarse únicamente en un uso aislado de IA generativa. Ya que el objetivo del trabajo no consiste simplemente en solicitar a un modelo que redacte escenarios, sino en construir esa arquitectura capaz de procesar documentos, estructurar la información, transformarla a un formato como Gherkin, etc. Esto exige que exista la capacidad de decisión, y el uso de herramientas por parte de los agentes.

En este trabajo, la conexión que hacen los agentes con Kiwi TCMS mediante su API constituye una parte clave dentro del flujo, ya que la utilidad práctica está en transformar esa salida en un recurso operativo dentro del proceso real de pruebas.

Sin esta integración, la solución queda reducida a un ejercicio de generación textual al que ya está acostumbrado prácticamente todo el mundo.

Otro aspecto que justifica la propuesta es la necesidad de mantener coherencia y trazabilidad entre las distintas fases del proceso. El hecho de analizar la documentación, estructurar requisitos, transformarlos en escenarios y registrarlos posteriormente en una herramienta exige mantener la relación entre la información original y la salida generada. Por tanto, esta necesidad de gestionar el contexto, el estado y la consistencia hace que una arquitectura multiagente resulte muy adecuada para el problema que se aborda.

En cuanto al análisis de las herramientas existentes se puede afirmar que ninguna de ellas cubre de forma completa todas las necesidades del trabajo.

Herramientas como Selenium, Cypress, JUnit, TestNG o Playwright son muy útiles en la automatización y ejecución de pruebas, pero no resuelven la fase previa de análisis documental y construcción estructurada de escenarios.

Por su parte, plataformas de testing basadas en IA, como Testim, Applitools, Mabl o Functionize, aportan capacidades avanzadas de asistencia y automatización, pero no están necesariamente orientadas a lo que se plantea en este trabajo, ni tampoco garantizan una adaptación directa al contexto del proyecto.

Finalmente, **la elección de Google ADK** se debe a que en un principio dentro de la empresa se realizaron pruebas con este framework y dio buenos resultados, y cabe destacar que Google no llega a ser partnership de CIC, pero le ofrece soporte de primer nivel en caso de necesitar ayuda, por tanto, el uso de esta herramienta encaja perfectamente.

Es cierto que se podrían usar otros frameworks como LangGraph pero debido a estas causas se utiliza Google ADK. A partir de esta decisión, se puede decir que Google ADK ofrece unas capacidades muy apropiadas que permiten construir la arquitectura multiagente y también flujos estructurados, por lo que toda esta suma de ventajas lo convierte en una opción adecuada para la arquitectura propuesta.

2.2. SOLUCIÓN PROPUESTA

Este capítulo presenta el punto de partida del proyecto.

En el segundo capítulo se realizará el análisis correspondiente a este proyecto con su respectivo modelo de dominio y disciplina de requisitos.

Debido a todo lo comentado en los otros apartados, la solución propuesta consiste en diseñar una arquitectura basada en agentes de Inteligencia Artificial con ADK Google utilizando el modelo de OpenAI, para que apoye distintas fases del proceso de testing de QA de CIC Consulting Informático.

La idea central es estructurar el flujo de trabajo en tareas especializadas.

Solución Propuesta:

Un agente puede encargarse de analizar el DRF (Documento Requisitos Funcionales) y el DDS (Documentos Diseño Sistema) y con ellos extraer requisitos o casos de uso relevantes; otro procesará la salida generada y transformará esa información a casos de pruebas, como Gherkin (escenarios (Given, When, Then, And)); y otro puede coger esos casos de prueba en dicho lenguaje para integrarlos dentro KiwiTCMS a través de su API.

La propuesta se plantea como un prototipo controlado, de manera que sea posible comprobar su viabilidad técnica y valorar su utilidad dentro de un contexto real.

2.3. OBJETIVOS GENERAL Y ESPECÍFICOS

Este apartado recoge los objetivos que se pretenden conseguir con la realización del TFG. Los objetivos se han de dividir en objetivo general y objetivos específicos, correspondientes con cada una de las etapas de desarrollo de la solución propuesta.

Objetivo general

El objetivo general de este Trabajo de Fin de Grado es generar escenarios Gherkin a partir de documentación e integrarlos en Kiwi TCMS mediante su API, mediante agentes de Inteligencia Artificial con el modelo de OpenAI.

Objetivos específicos

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1) Elaborar el modelo de dominio y definir la disciplina de requisitos del sistema.

2) Diseñar los agentes de IA necesarios para que el flujo funcione de manera óptima.

3) Desarrollar un prototipo funcional controlado.

4) Evaluar las ventajas y aportaciones del prototipo.

Alcance del trabajo

El alcance de este TFG se concreta en los siguientes puntos:

1) Analizar y documentar (modelo de dominio y disciplina de requisitos) el proceso de testing de este trabajo.

2) Diseñar una arquitectura basada en agentes de IA con Google ADK y con un modelo de OpenAI para el prototipo funcional.

3) Implementar un prototipo funcional controlado con agentes de IA que automatice una parte representativa del proceso, desde el análisis de documentos DDS/DRF y la extracción de casos de uso y requisitos hasta la generación de escenarios Gherkin y su registro en Kiwi TCMS mediante API.

4) Validar el prototipo en un entorno experimental controlado.

5) Evaluar las ventajas obtenidas a partir del modelo propuesto.

En este trabajo se busca demostrar que los agentes de IA pueden actuar como herramienta de apoyo dentro del proceso de QA, no se pretende automatizar en este TFG el 100% del flujo de trabajo, es decir, no se van a realizar test e2e, solo se llegará hasta los escenarios en Kiwi TCMS.

Y, por último, es necesario decir que la arquitectura propuesta intentará ser planteada de forma que pueda servir como base para futuros desarrollos dentro de la empresa.

2.4. ESTRUCTURA DEL TRABAJO

Este apartado expone de manera sucinta la estructura final del trabajo, los apartados que contiene y la información recogida en cada uno de dichos apartados.

Metodología - Rational Unified Process (RUP)

Se utilizará RUP como marco metodológico, es un marco de trabajo (framework) para el desarrollo de software, iterativo, incremental y centrado en la arquitectura.

Estructura del Proyecto

La metodología que se va a utilizar (RUP) estructura el desarrollo en cuatro fases:

- **Inicio**: definición del problema y alcance.

- **Elaboración**: análisis, modelo de dominio y disciplina de requisitos.

- **Construcción**: diseño e implementación del prototipo controlado.

- **Transición**: validación y evaluación del prototipo.

3. REQUERIMIENTOS Y REQUISITOS

Este capítulo debe plantear el levantamiento de los requisitos de usuario que gobierna las siguientes etapas de la ejecución de la solución presentada en el TFG.

En el contexto del grado de Ingeniería Informática aquí se recogen los requisitos de usuario, especificación de requerimientos, requerimientos funcionales y no funcionales, entre otros, adecuadamente expresados de acuerdo a la metodología aprendida a lo largo del grado.

4. ANÁLISIS Y DISEÑO

El capítulo Análisis y Diseño toma los requisitos descritos en el capítulo anterior y define los elementos que componen la solución presentada en el TFG. En este capítulo se presentan, entre otros, diagramas de comportamiento (actividades, casos de uso, secuencia y comunicación), diagramas de arquitectura, despliegue, paquetes, objetos, clases de diseño, DER y modelo físico.

.

5. DESCRIPCIÓN DE LA SOLUCIÓN PROPUESTA

Este capítulo presenta de la manera más detallada posible la solución desarrollada a partir de los elementos expuestos en los capítulos anteriores.

6. CONCLUSIONES

El capítulo Conclusiones expone una breve reflexión sobre la solución presentada, los resultados obtenidos en su aplicación, así como una alusión a las limitaciones que ha sufrido el estudio por diversos motivos (restricciones de espacio, poca accesibilidad a ciertos tipos textuales, etc.), recomendaciones y posibles líneas de investigación que podrían mejorar o incrementar el alcance del estudio llevado a cabo en el TFG.

7. REFERENCIAS BIBLIOGRÁFICAS

El capítulo Referencias bibliográficas comprenderá todas las obras citadas a lo largo del cuerpo del trabajo. Este apartado no se contabilizará en el cómputo de páginas.

La norma de citación y referencias que se habrá de seguir es APA.

La disposición de las referencias bibliográficas se hará por estricto orden alfabético:

Amazon Web Services. (s. f.). ¿Qué es un modelo de lenguaje grande (LLM)?

https://aws.amazon.com/es/what-is/large-language-model/

Applitools. (s. f.). Applitools documentation.

https://applitools.com/docs/

CIC Consulting Informático. (s. f.). CIC Consulting Informático.

https://www.cic.es/

CrewAI. (s. f.). CrewAI documentation.

https://docs.crewai.com/

Cypress. (s. f.). Cypress documentation.

https://docs.cypress.io/

Functionize. (s. f.). Enterprise AI test automation platform.

https://www.functionize.com/

Google. (s. f.). Agent Development Kit (ADK): Technical overview.

https://google.github.io/adk-docs/

JUnit Team. (s. f.). JUnit 5 user guide.

https://junit.org/junit5/docs/current/user-guide/

Kiwi TCMS. (s. f.). Kiwi TCMS documentation.

https://kiwitcms.readthedocs.io/

LangChain. (s. f.). LangGraph overview.

https://docs.langchain.com/langgraph/

mabl. (s. f.). Unified test automation platform for web, mobile, and APIs.

https://www.mabl.com/

Microsoft. (s. f.). AutoGen documentation.

https://microsoft.github.io/autogen/

NihadMemmedli. (s. f.). quorvex_ai. GitHub.

https://github.com/NihadMemmedli/quorvex_ai

Playwright Team. (s. f.). Playwright: End-to-end testing for modern web apps.

https://playwright.dev/

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

Selenium Project. (s. f.). The Selenium browser automation project.

https://www.selenium.dev/

Sharma, A., & Kumar, R. (2024). AI-driven software testing: Opportunities and challenges. Journal of Software Testing Research, 12(2), 45–60.

Testim. (s. f.). Automated UI and functional testing.

https://www.testim.io/

TestNG. (s. f.). TestNG documentation.

https://testng.org/

Wang, L., Zhang, Y., & Li, H. (2024). Large language models in software engineering: A survey. ACM Computing Surveys, 56(8), 1–36.

8. ANEXOS

En este capítulo se deberán incluir los materiales complementarios que se consideran inadecuados incluir en el cuerpo del trabajo por razones de espacio o de continuidad, pero se consideran relevantes para el lector potencial del TFG y su evaluación.

Los anexos que se incluyan en este apartado aparecerán por orden de aparición en el texto principal, y deberán estar nombrados de manera adecuada para una identificación rápida y eficaz.

Este apartado no se contabilizará en el cómputo de páginas.